Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту



Звіт

про виконання лабораторних та практичних робіт блоку № 4

На тему: «Двовимірні Масиви. Вказівники та Посилання. Динамічні масиви. робота з масивами та алгоритми обробки.»

з дисципліни: «Основи програмування»

до:

Лабораторної Роботи №4
Лабораторної Роботи №5
Алготестер Лабораторної Роботи №2
Алготестер Лабораторної Роботи №3
Практичних Робіт до блоку №4

Виконала:
Студентка групи ШІ-13
Сєрова Дарія Віталіївна

Львів 2026

Тема роботи:

Двовимірні Масиви. Вказівники та Посилання. Динамічні масиви. Робота з масивами та алгоритми обробки.

Мета роботи:

Ознайомитися з класами пам’яті (статична, динамічна, стек) та механізмами адресації в мові C++. Засвоїти синтаксис і принципи використання вказівників, посилань, арифметики вказівників, а також оголошення та обробки двовимірних і динамічних масивів. Навчитися динамічно виділяти та вивільняти пам’ять за допомогою операторів new та delete (delete[]), реалізовувати алгоритми пошуку та сортування, будувати блок-схеми, оцінювати складність задач та оформлювати й публікувати результати роботи в репозиторії GitHub відповідно до вимог 4-ї ітерації.

Теоретичні відомості:

1. Теоретичні відомості з переліком важливих тем:

- Тема №1: Класи пам'яті у C++
- Тема №2: Вступ до Масивів і Вказівників
- Тема №3: Вказівники та Посилання у функціях та алгоритмах
- Тема №4: Двовимірні Масиви
- Тема №5: Динамічні Масиви та Управління Пам'яттю
- Тема №6: Алгоритми обробки та робота з масивами

2. Індивідуальний план опрацювання теорії:

- Тема №1: Класи пам'яті у C++
    - Джерела Інформації
        - Практичні заняття з Основ Програмування
        - Лекції О. Пшеничного
        - W3Schools: C++ Scope & Memory Access
    - Що опрацьовано:
        - Вивчено концепції статичної та динамічної пам'яті, поняття стеку та купи (heap). Опрацьовано принципи виділення та вивільнення пам'яті під час виконання програми, а також область видимості та час життя змінних.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.26

- Тема №2: Вступ до Масивів і Вказівників
    - Джерела Інформації:
        - Лекції О. Пшеничного
        - W3Schools: C++ Pointers
        - W3Schools: C++ Arrays
    - Що опрацьовано:
        - Опрацьовано фундаментальні відмінності між статичними та динамічними масивами. Вивчено основи роботи з вказівниками (адресація, розіменування) та посиланнями. Розглянуто базовий взаємозв'язок між масивами та вказівниками.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.26

- Тема №3: Вказівники та Посилання
    - Джерела Інформації:
        - Лекції О. Пшеничного
        - Практичні заняття
        - W3Schools: C++ Dereference & Pointers
        - W3Schools: C++ Pass by Reference
    - Що опрацьовано:
        - Детально вивчено арифметику вказівників та їх використання для доступу до елементів масиву. Опрацьовано відмінності між передачею параметрів у функції за посиланням та за вказівником. Розглянуто основи побудови складних структур даних за допомогою вказівників.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.26
- Тема №4 Двовимірні Масиви
    - Джерела Інформації:
        - Практичні заняття.
        - W3Schools: C++ Multi-Dimensional Arrays
    - Що опрацьовано:
        - Вивчено оголошення та ініціалізацію двовимірних (багатовимірних) масивів. Опрацьовано використання

вкладених циклів for для обходу та маніпуляції матрицями, а також особливості передачі двовимірних масивів у функції.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.26
- Тема №5 Динамічні Масиви
    - Джерела Інформації:
        - Практичні та лабораторні заняття
        - W3Schools: C++ Dynamic Memory (new / delete)

    - Що опрацьовано:
        - Засвоєно принципи динамічного виділення та управління пам'яттю за допомогою операторів new та delete / delete[]. Реалізовано створення масивів зі змінною розмірністю в часі виконання (runtime) та їх безпечну передачу до функцій.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.2026

- Тема №6 Алгоритми обробки та робота з масивами
    - Джерела Інформації:
        - Лекції Пшеничного
        - W3Schools: C++ Algorithms & Array Manipulation

    - Що опрацьовано:
        - Досліджено базові алгоритми пошуку (лінійний, бінарний) та сортування в масивах. Застосовано комбінації циклів і умовних операторів для розв'язання комплексних практичних задач та обробки великих наборів даних.
    - Статус: Ознайомлена
    - Початок опрацювання теми: 28.10.26
    - Звершення опрацювання теми: 31.10.2026

Виконання роботи:

1. Опрацювання завдання та вимог до програм та середовища:

Завдання №1 Lab# programming: Lab 4
Варіант завдання: 10

10.

1) Сформувати одновимірний масив цілих чисел, використовуючи генератор випадкових чисел.
2) Роздрукувати отриманий масив.
3) Знищити 5 перших елементів масиву.
4) Додати в кінець масиву 3 нових елементи.
5) Роздрукувати отриманий масив.

Рис 1. Умова задачі lab 4

Деталі завдання:

1. При виконанні роботи використовуються статичні масиви. Для організації статичних масивів із псевдозмінними межами необхідно оголосити масив досить великої довжини, наприклад, 100 елементів: int N=100; int a[N];

Потім користувач вводить реальну довжину масиву (не більше N) і працює з масивом тієї довжини, що він сам вказав. Інші елементи (хоча пам'ять під них і буде виділена) не розглядаються.

2) При зменшенні або збільшенні довжини масиву необхідно змінювати його реальну довжину.

Завдання №2 Lab# programming: Lab 5

Варіант завдання: 10

10. Написати функцію, що перевіряє чи є від’ємні елементи в зазначеному рядку двовимірного масиву. Знищити з масиву всі рядки з від’ємними елементами, знищений рядок заповнюється 0 і переноситься в кінець масиву.

Рис 2. Умова задачі lab 5

Завдання №3 Lab# programming: Algotester Lab 2

У вас є дорога, яка виглядає як N чисел.

Після того як ви по ній пройдете - вашу втому можна визначити як різницю максимального та мінімального елементу.

Ви хочете мінімізувати втому, але все що ви можете зробити - викинути одне число з дороги, тобто забрати його з масиву.

В результаті цієї дії, яку мінімальну втому ви можете отримати в кінці дороги?

Input

У першому рядку ціле число N - кількість чисел

У другому рядку масив r, який складається з N цілих чисел

Output

Єдине ціле число m - мінімальна втома, яку можна отримати

Constraints

$1 \leq N \leq 10^5$

$0 \leq r_i \leq 10^5$

Рис 3. Умова задачі algotester lab 2

Завдання №4 Lab# programming: Algotester Lab 3

Вам дана стрічка s.

Ваше завдання зробити компресію стрічки, тобто якщо якась буква йде більше одного разу підряд у стрічці замінити її на букву + кількість входжень підряд.

Input

У першому рядку стрічка S

Output

Стрічка $S_{compressed}$

Constraints

$1 \leq |S| \leq 10^5$

Рис 3. Умова задачі algotester lab 3

Завдання №5 Practice# programming: Class Practice Task

Редактор монохромного зображення

Задача

Ви пишете простий редактор зображень у відтінках сірого. Зображення задається прямокутною таблицею чисел від 0 до 9, де 0 це чорний піксель, а 9 найсвітліший. Програма показує таку таблицю як малюнок із символів і дозволяє застосовувати до неї перетворення.

Розміри зображення заздалегідь невідомі: їх задає користувач під час роботи. Тому пам’ять під зображення виділяється динамічно, а після повороту на дев’яносто градусів висота і ширина міняються місцями, і масив доводиться виділяти заново.

Обмеження: сторонні бібліотеки використовувати заборонено. Ввід і вивід лише через cin та cout.

Мета задачі

- Навчитися виділяти й звільняти динамічну пам’ять під двовимірні дані операторами new та delete.

- Навчитися обходити двовимірний масив вкладеними циклами, зокрема зі зміною порядку індексів.
- Навчитися передавати дані у функції вказівниками та повертати кілька результатів через параметри-посилання.
- Навчитися проєктувати алгоритм блок-схемою до написання коду.

Вхідні дані

Програма спочатку зчитує розміри та самі пікселі, і лише після цього показує меню:

№	Значення	Позн.	Формат вводу
1	Висота зображення, пікселів	`rows`	Ціле число, напр. 4, від 1 до 20
2	Ширина зображення, пікселів	`cols`	Ціле число, напр. 5, від 1 до 20
3	Яскравість пікселів	`image`	rows рядків по cols чисел від 0 до 9

Зображення зберігається у динамічному двовимірному масиві типу `int **`, виділеному за розмірами, які щойно ввів користувач.

Якщо висота, ширина або значення яскравості виходять за допустимі межі, повідомте про це користувачу і завершіть роботу програми.

Для показу зображення використовується палітра з десяти символів, де позиція символу відповідає яскравості: `" .:-=+*#%@"`. Нульовий піксель виводиться пробілом, дев’ятка символом at.

Меню

Меню повторюється, доки користувач не вибере вихід. Кожне перетворення застосовується до поточного стану зображення, а не до початкового.

Пункт	Операція
`1`	Показати зображення
`2`	Статистика яскравості
`3`	Інвертувати яскравість
`4`	Віддзеркалити по горизонталі
`5`	Повернути на 90 градусів праворуч
`0`	Вийти

Деталі операцій

1. Показати зображення

- Спершу виводиться таблиця чисел: рядок за рядком, числа через пробіл.
- Далі те саме зображення виводиться символами палітри, по одному символу на піксель.
- Під зображенням показуються його поточні розміри, бо після повороту вони змінюються.

2. Статистика яскравості

- Найтемніший і найсвітліший пікселі разом з їхніми координатами. Обидві пари координат повертає одна функція через параметри-посилання: `void findExtremes(int **img, int rows, int cols, int &minRow, int &minCol, int &maxRow, int &maxCol)`.
- Якщо однакових значень кілька, береться перше в порядку обходу по рядках.
- Середня яскравість усього зображення та середня яскравість кожного рядка окремо.
- Обхід рядка у цьому пункті виконується через вказівник: `int *row = img[i];` і звернення виду `*(row + j)`. Індексація виду `img[i][j]` тут не застосовується.

3. Інвертувати яскравість

- Кожен піксель замінюється на різницю між дев'яткою та його значенням: темне стає світлим і навпаки.
- Перетворення виконується на місці, у тому самому масиві, без виділення нової пам'яті.
- Координати пікселів не змінюються, розміри зображення залишаються тими самими.

4. Віддзеркалити по горизонталі

- Перший стовпець міняється місцями з останнім, другий із передостаннім і так далі до середини.
- Обмін двох пікселів виконує окрема функція, яка приймає їх адреси: `void swapPixels(int *a, int *b)`. Усередині значення обмінюються через розіменування.
- Перетворення виконується на місці, розміри зображення не змінюються.

5. Повернути на 90 градусів праворуч

- Після повороту висота і ширина міняються місцями, тому перетворення на місці неможливе.
- Виділяється новий динамічний масив розмірами `cols` на `rows`, у нього переносяться пікселі за правилом «перший рядок початкового зображення стає останнім стовпцем нового».
- Поворот виконує функція `int** rotateRight(int** img, int rows, int cols)`: вона виділяє новий масив, переносить у нього пікселі, звільняє старий і повертає адресу нового. Виклик має вигляд `img = rotateRight(img, rows, cols);`, після чого `rows` і `cols` міняються місцями.
- Порядок дій має значення: якщо звільнити старий масив до перенесення, зображення буде втрачено, а якщо не звільнити його зовсім, станеться витік пам'яті.

6. Звільнення пам’яті

- Перед завершенням програми звільняється кожен рядок у циклі, а потім сам масив вказівників.
- Кожному `new` відповідає рівно один `delete[]` відповідної форми. Після звільнення вказівники обнуляються через `nullptr`.
- Витік пам’яті вважається дефектом нарівні з неправильним результатом.

Вимоги

1. `new` / `delete[]`: пам’ять під зображення виділяти динамічно за розмірами, які ввів користувач, і перед виходом повністю звільняти: спершу кожен рядок, потім масив вказівників.
2. Вкладені цикли: обходити двовимірний масив для виводу, статистики й перетворень; для повороту обхід зі зміною порядку індексів.
3. Вказівники: у статистиці обходити рядок через `*(row + j)`, а обмін пікселів робити функцією `swapPixels(int *a, int *b)`.
4. Параметри-посилання: координати найтемнішого й найсвітлішого пікселів повертати однією функцією `findExtremes`.
5. Функції: кожен пункт меню окремою функцією, зображення передається в неї вказівником разом із розмірами.
6. Поворот без витоків: у `rotateRight` спершу перенести пікселі в новий масив і лише потім звільнити старий; перед виходом із програми звільнені вказівники обнулити через `nullptr`.
7. `do while`: після кожної операції повертати користувача в меню і працювати, доки він не вибере вихід.
8. Блок-схема: **обов’язкова**
9. Валідація: перевіряти всі вхідні дані, зокрема неправильний пункт меню та нечислові символи, правила визначити самостійно на основі попередніх задач.

Завдання №6 Practice# programming: Self Practice Task 1

Решта

Обмеження: 2 сек., 256 МіБ

Ні для кого не секрет, що Марічка для Зеника є сенсом його життя. Він заради неї ладен на все й нічого не пошкодує для того, щоб Марічка була щасливою.

Одного разу Зеник вирішив придбати Марічці дороге намисто в розкішному магазині в самі-сінькому центрі Львова. Біля каси він гонорово витягнув купюру вартістю 500 гривень. Продавець має багато купюр таких вартостей: 1, 2, 5, 10, 20, 50, 100, 200 та 500 гривень. Він дасть Зеникові решту таким чином, щоб використати якомога меншу кількість купюр. Намисто коштує n гривень.

Вам необхідно визначити, яку кількість купюр отримає Зеник від продавця.

Вхідні дані

Єдиний рядок містить ціле число n — ціну намиста в гривнях.

Вихідні дані

У єдиному рядку виведіть ціле число — кількість купюр, що отримає Зеник.

Обмеження

$1 \le n \le 500$.

Рис 4. Умова задачі self practice task 1

Завдання №7 Practice# programming: Self Practice Task 2

Запити на матриці

Обмеження: 2 сек., 256 МіБ

Вам задано прямокутну матрицю, що складається з n рядків та m стовпців. a_{ij} — j-ий елемент в i-ому рядку матриці. Рядки нумеруються зверху вниз, елементи в рядках нумеруються зліва направо.

Вам потрібно обробити q запитів. Запити бувають двох типів:

- `1` x_1 y_1 x_2 y_2 h w – поміняти місцями дві підматриці з h рядків та w стовпців.
  (x_1, y_1) — верхній лівий кут першої підматриці, (x_2, y_2) — верхній лівий кут другої підматриці.
  Гарантується, що підматриці не перетинаються.
- `2` x y — вивести значення a_{xy}.

Вхідні дані

У першому рядку задано два цілі числа n та m — кількіть рядків та стовпців матриці.

У наступних n рядках задано по m цілих чисел a_{ij} — елементи матриці.

У наступному рядку — ціле число q — кількість запитів.

Кожен із наступних q рядків починається з числа t — типу запиту.

Якщо $t = 1$, тоді після нього задано шість цілих чисел x_1, y_1, x_2, y_2, h, w — координати верхнього лівого кута першої підматриці, координати верхнього лівого кута другої підматриці та розміри підматриці. Якщо $t = 2$, то за ним задано два цілих числа x, y — координати шуканого елемента матриці.

Вихідні дані

Для кожного запити другого типу виведіть один рядок зі значенням шуканого елемента.

Обмеження

$1 \le n, m, q \le 2 \cdot 10^3$,

$1 \le a_{ij} \le 10^9$.

Для кожного запиту:

$t \in \{1, 2\}$,

$1 \le x, x_1, x_2, h \le n$,

$1 \le y, y_1, y_2, w \le m$.

Рис 5. Умова задачі self practice task 2

Завдання №8 Practice# programming: Self Practice Task 3

Зуби

Обмеження: 2 сек., 256 МіБ

Мале Бісеня любить гострити зуби. А Зла Тітонька любить до нього підходити і питатися: «Що, зуби гостриш?». Бісеняті таке не дуже подобається, тому воно придумало робити таке.

У Малого Бісеняти є n зубів. Кожен зуб має коефіцієнт загостреності a_i. Також існує межа загостреності k. Якщо коефіцієнт загостреності певного зуба є більшим чи рів межі загостреності, то такий зуб вважається загостреним.

Мале Бісеня хоче наступного разу, коли Зла Тітонька його щось запитає, показати їй якнайбільше загострених зубів, що розташовані поспіль.

Допоможіть Малому Бісеняті дізнатися, скільки найбільше зубів воно зможе показати.

Вхідні дані

У першому рядку задані два цілих числа n та k — кількість зубів та межа загостреності відповідно.

В другому рядку задано n цілих чисел a_i — коефіцієнти загостреності зубів.

Вихідні дані

Єдине ціле число — відповідь на задачу.

Обмеження

$1 \le n \le 10^5$,

$1 \le k \le 10^9$,

$1 \le a_i \le 10^9$.

Рис 6. Умова задачі self practice task 3

Завдання №9 Result Documentation Report and Outcomes Placement Activities (Docs and Programs on GitHub)

- Деталі завдання: оформити звіт стосовно проробленої роботи та поділитися ним і кодами програм через Github використовуючи git команди.

Завдання №10 Results Evaluation and Release

- Деталі завдання: захистити звіт по 4 епіку та вивчений матеріал.

2. Дизайн та планована оцінка часу виконання завдань:

Програма №1 Lab# programming: Lab 4

- Планований час на реалізацію: 35хв

Програма №2 Lab# programming: Lab 5

- Планований час на реалізацію: 40 хв

Програма №3 Lab# programming: Algotester Lab 2

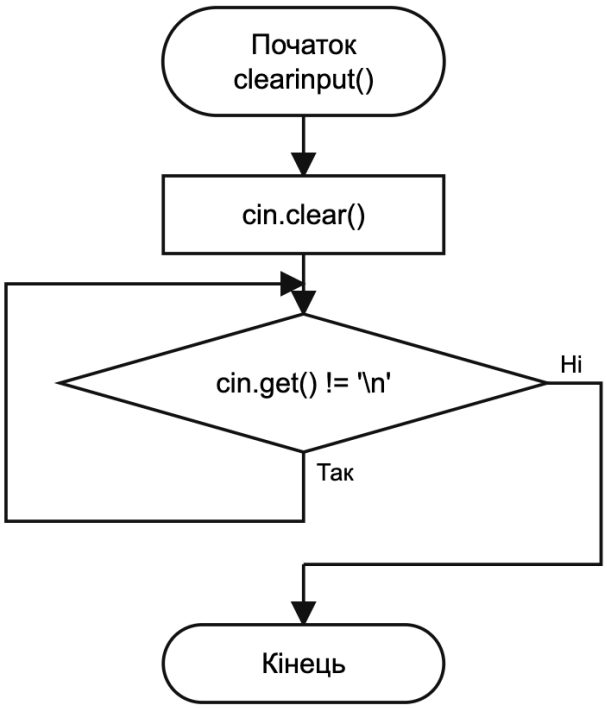
- Планований час на реалізацію: 50 хв

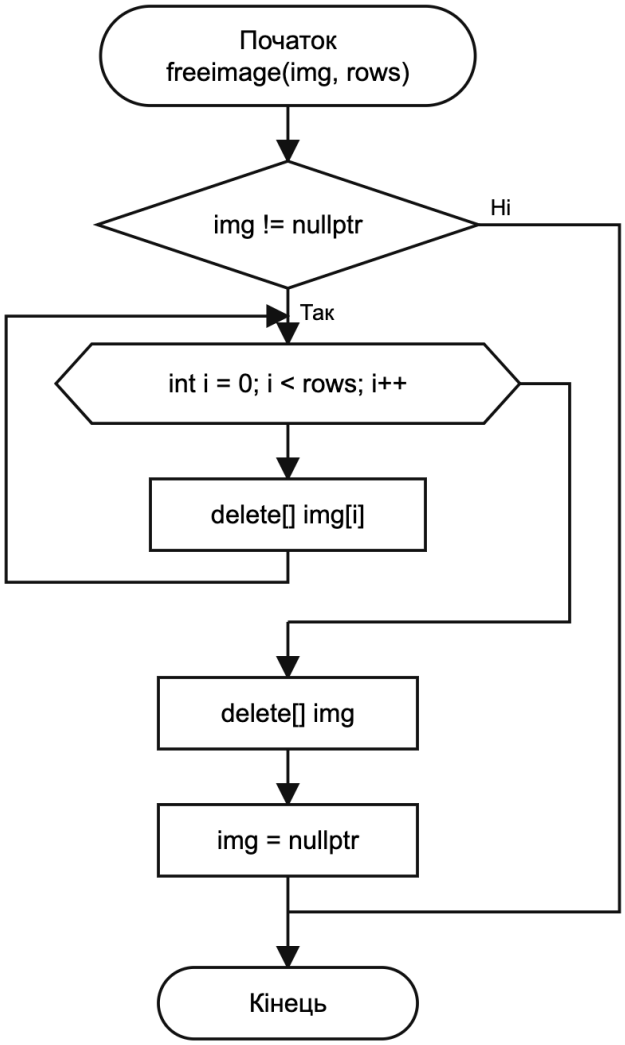
Програма №4 Lab# programming: Algotester Lab 3

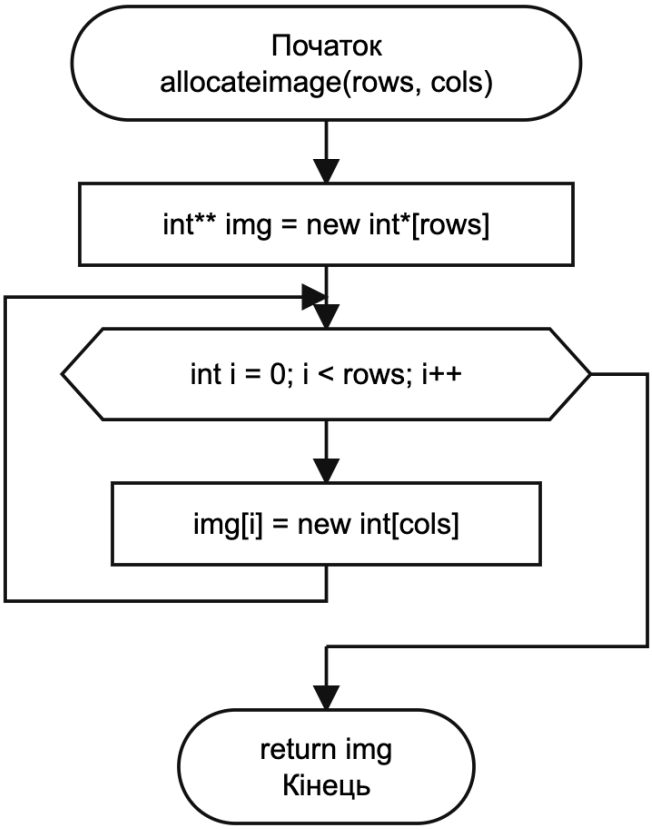
- Планований час на реалізацію: 50 хв

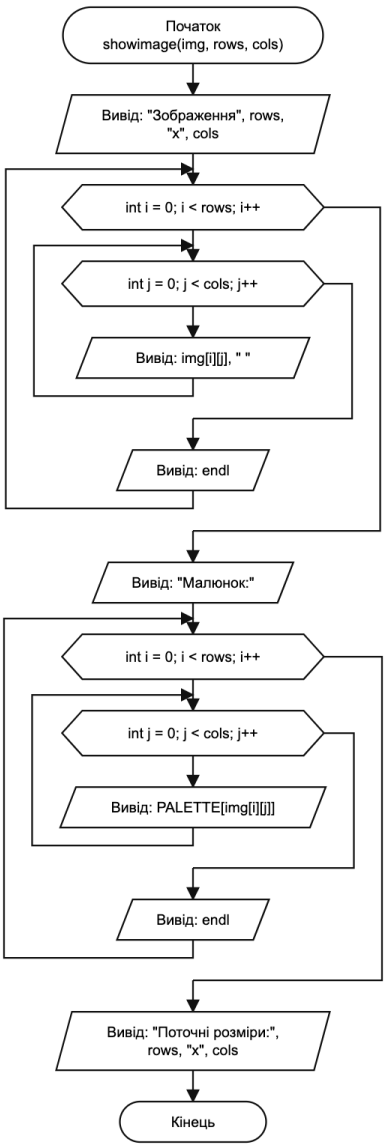
Програма №5 Practice# programming: Class Practice Task

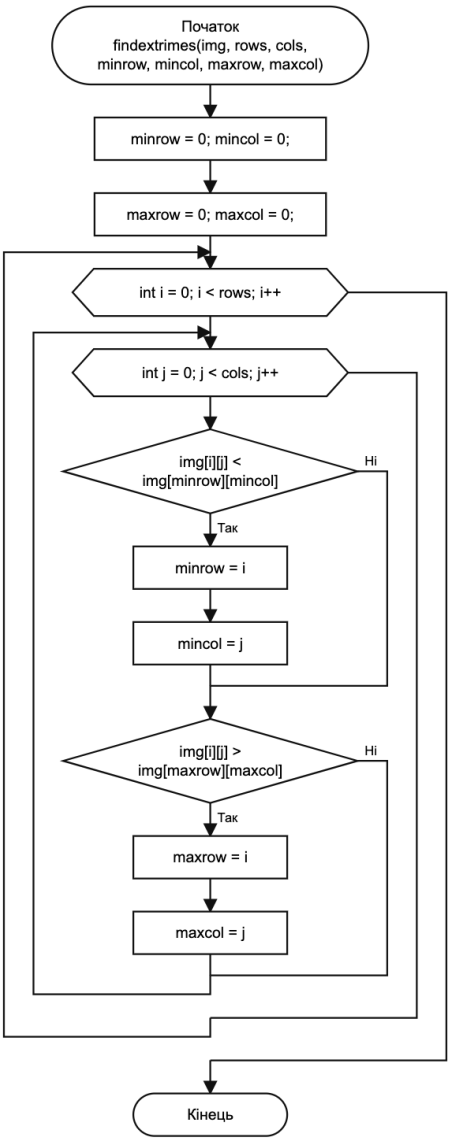
- Блок-схема:

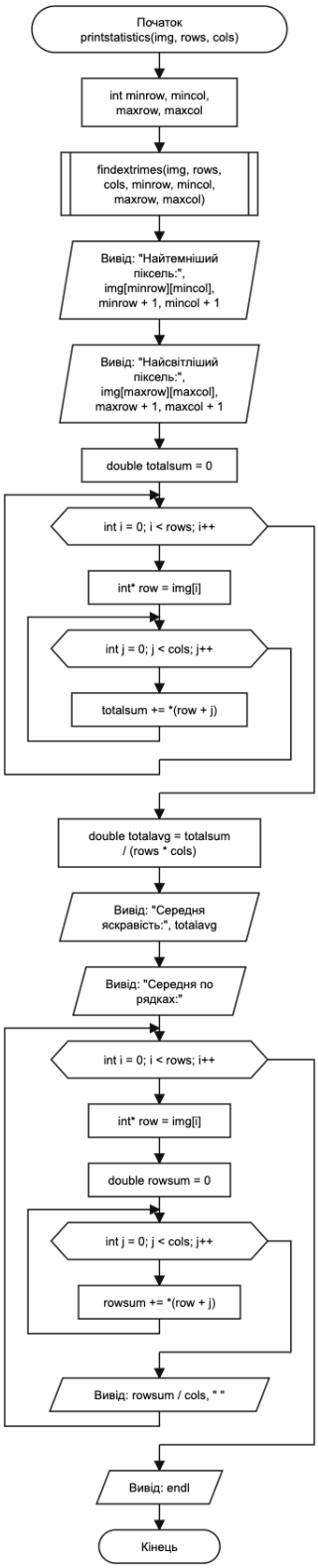


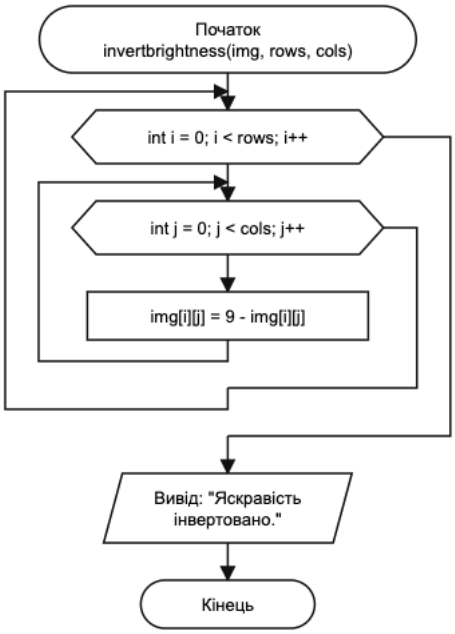

Початок
freeimage(img, rows)
img != nullptr
Ні
Так
int i = 0; i < rows; i++
delete[] img[i]
delete[] img
img = nullptr
Кінець

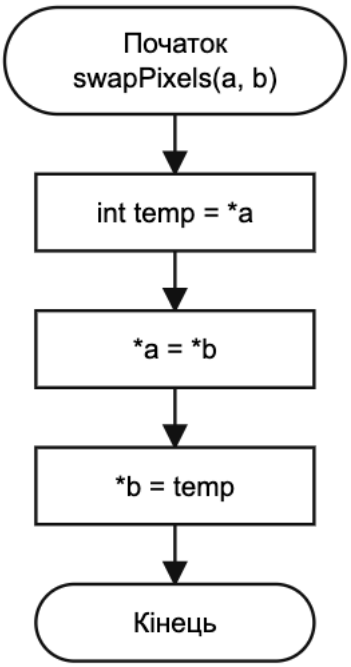

Початок
allocateimage(rows, cols)
int** img = new int*[rows]
int i = 0; i < rows; i++
img[i] = new int[cols]
return img
Кінець

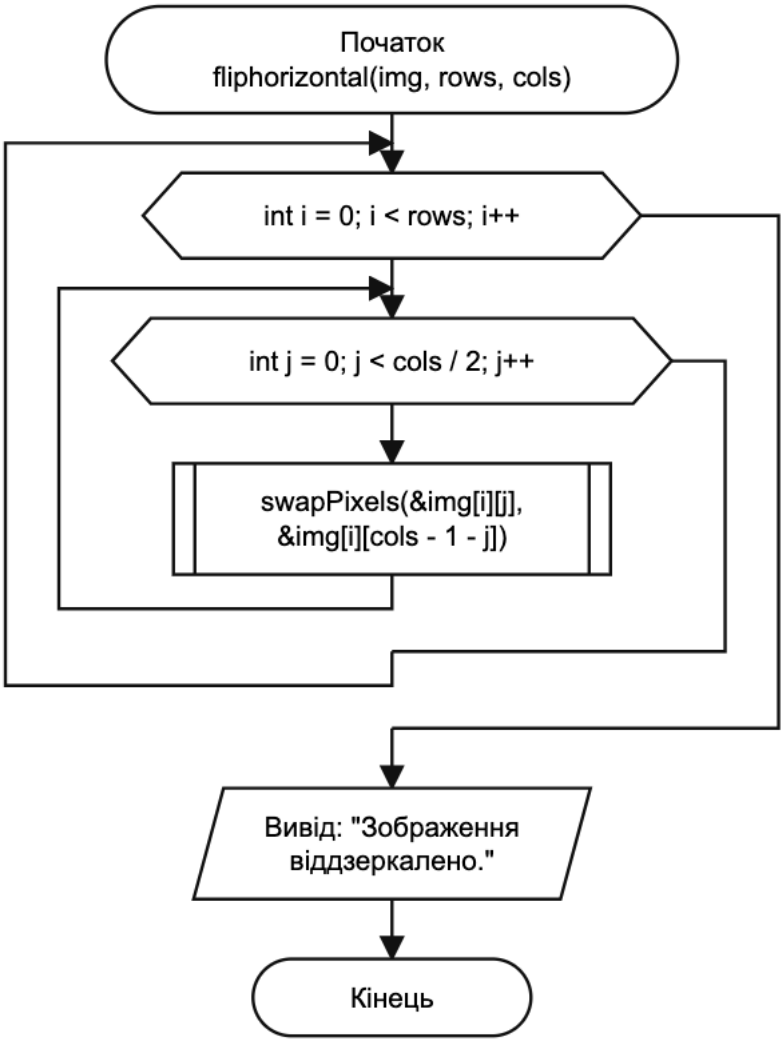

Початок
showimage(img, rows, cols)
Вивід: "Зображення", rows,
"x", cols
int i = 0; i < rows; i++
int j = 0; j < cols; j++
Вивід: img[i][j], " "
Вивід: endl
Вивід: "Малюнок:"
int i = 0; i < rows; i++
int j = 0; j < cols; j++
Вивід: PALETTE[img[i][j]]
Вивід: endl
Вивід: "Поточні розміри:",
rows, "x", cols
Кінець

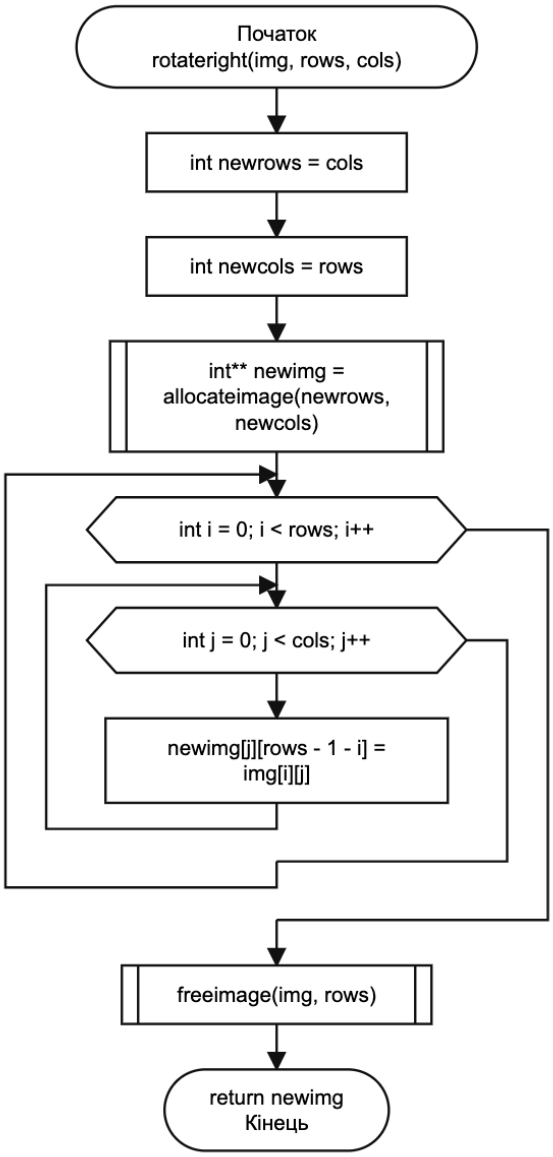

Початок
findextrimes(img, rows, cols,
minrow, mincol, maxrow, maxcol)
minrow = 0; mincol = 0;
maxrow = 0; maxcol = 0;
int i = 0; i < rows; i++
int j = 0; j < cols; j++
img[i][j] <
img[minrow][mincol]
Ні
Так
minrow = i
mincol = j
img[i][j] >
img[maxrow][maxcol]
Ні
Так
maxrow = i
maxcol = j
Кінець

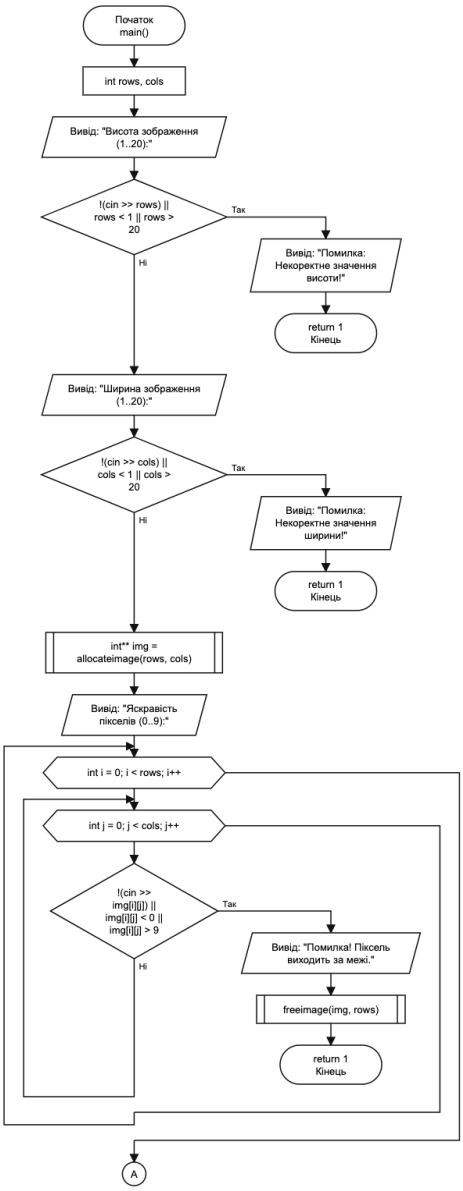

Початок
printstatistics(img, rows, cols)
int minrow, mincol,
maxrow, maxcol
findextrimes(img, rows,
cols, minrow, mincol,
maxrow, maxcol)
Вивід: "Найтемніший
піксель:",
img[minrow][mincol],
minrow + 1, mincol + 1
Вивід: "Найсвітліший
піксель:",
img[maxrow][maxcol],
maxrow + 1, maxcol + 1
double totalsum = 0
int i = 0; i < rows; i++
int* row = img[i]
int j = 0; j < cols; j++
totalsum += *(row + j)
double totalavg = totalsum
/ (rows * cols)
Вивід: "Середня
яскравість:", totalavg
Вивід: "Середня по
рядках:"
int i = 0; i < rows; i++
int* row = img[i]
double rowsum = 0
int j = 0; j < cols; j++
rowsum += *(row + j)
Вивід: rowsum / cols, " "
Вивід: endl
Кінець


Початок
invertbrightness(img, rows, cols)
int i = 0; i < rows; i++
int j = 0; j < cols; j++
img[i][j] = 9 - img[i][j]
Вивід: "Яскравість
інвертовано."
Кінець


Початок
swapPixels(a, b)
int temp = *a
*a = *b
*b = temp
Кінець


Початок
fliphorizontal(img, rows, cols)
int i = 0; i < rows; i++
int j = 0; j < cols / 2; j++
swapPixels(&img[i][j],
&img[i][cols - 1 - j])
Вивід: "Зображення
віддзеркалено."
Кінець


Початок
rotateright(img, rows, cols)
int newrows = cols
int newcols = rows
int** newimg =
allocateimage(newrows,
newcols)
int i = 0; i < rows; i++
int j = 0; j < cols; j++
newimg[j][rows - 1 - i] =
img[i][j]
freeimage(img, rows)
return newimg
Кінець


Початок
main()
int rows, cols
Вивід: "Висота зображення
(1..20):"
!(cin >> rows) ||
rows < 1 || rows >
20
Так
Ні
Вивід: "Помилка:
Некоректне значення
висоти!"
return 1
Кінець
Вивід: "Ширина зображення
(1..20):"
!(cin >> cols) ||
cols < 1 || cols >
20
Так
Ні
Вивід: "Помилка:
Некоректне значення
ширини!"
return 1
Кінець
int** img =
allocateimage(rows, cols)
Вивід: "Яскравість
пікселів (0..9):"
int i = 0; i < rows; i++
int j = 0; j < cols; j++
!(cin >>
img[i][j]) ||
img[i][j] < 0 ||
img[i][j] > 9
Так
Ні
Вивід: "Помилка! Піксель
виходить за межі."
freeimage(img, rows)
return 1
Кінець
A

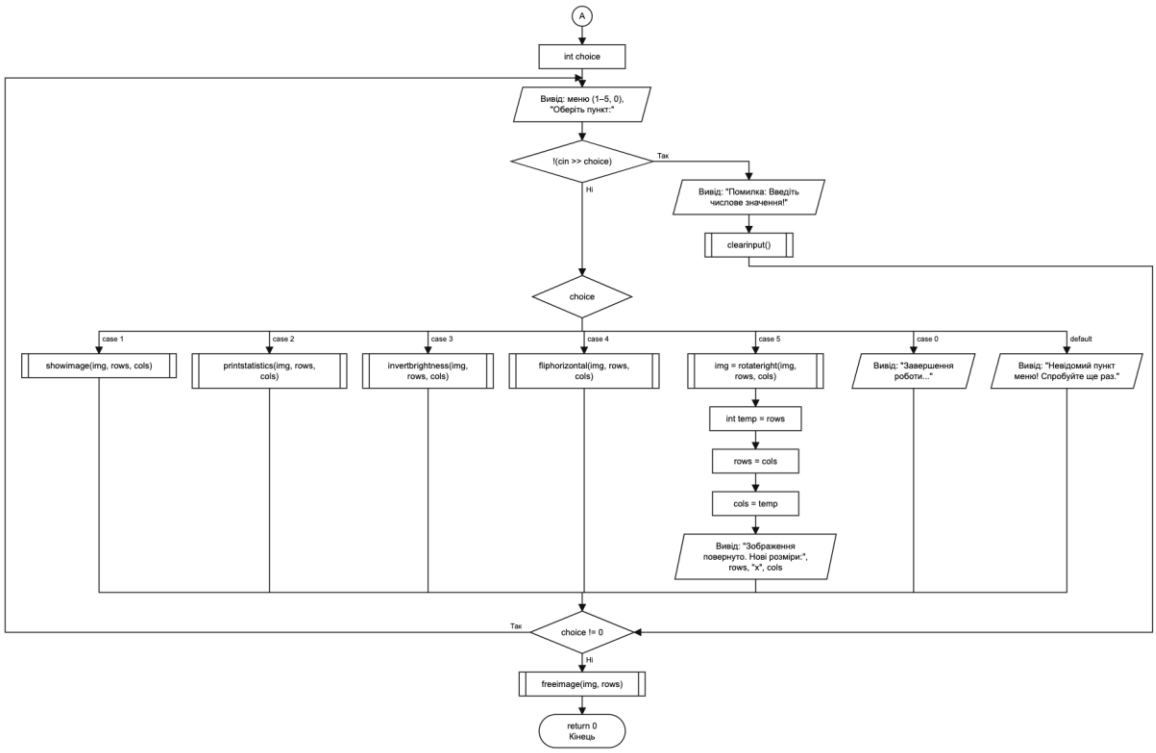


Рис 6 - 18 . Блок-схема до задачі Class Practice task

- Планований час на реалізацію: 120 хв

Програма №6 Practice# programming: Self Practice Task 1

- Планований час на реалізацію: 20 хв

Програма №7 Practice# programming: Self Practice Task 2

- Планований час на реалізацію: 20 хв

Програма №8 Practice# programming: Self Practice Task 3

- Планований час на реалізацію: 20 хв

4. Код програм з посиланням на зовнішні ресурси:

Завдання №1 Lab# programming: Lab 4

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

void printarr(const int* arr, int size){
    for (int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    srand(time(0)); // ініціалізація генератор випадкових числа

    int size; //початковий ромір
    cout << "Введіть розмір масиву: ";
    cin >> size;

    if(cin.fail()) return 0;

    if ( size < 5){
        cout << "Помилка, розмір масиву має бути не менше 5." << endl ;
        return 0;
    }

    int* arr = new int[size];
    // заповнюєм масив
    for ( int i = 0; i < size; i++){
        arr[i] = rand()%100 + 1; //діапазон від 1 до 100
    }

    // початковий масив
    cout << "Початковий масив (" << size << " елементів):";
    printarr(arr, size);

    // знищуєм 5 перших ел
    int k = 5;

    int newSize = size - k;
    int* temp = new int[newSize];        // копіюєм ел починаючи з 5 індексу
    for (int i = 0; i < newSize; i++){
        temp[i] = arr[i + k];
    }
```

```
46      delete[] arr;
47      arr = temp; // перенаправляємл arr на новий масив temp(адреса temp)
48      size = newSize;
49
50      cout << "Після видалення перших " << k << " елементів: ";
51      printarr (arr, size);
52
53      //додаємо 3 ел
54      int add_k = 3;
55      newSize += add_k;
56      temp = new int[newSize];
57      // копіюєм ел
58      for (int i = 0; i < size; i++){
59          temp[i] = arr[i];
60      }
61      // додаємо 3 випадкові
62      for (int i = size; i < newSize; i ++){
63          temp[i] = rand() % 100 + 1;
64      }
65
66      delete[] arr;
67      arr = temp;
68      size = newSize;
69
70      cout << "Після додавання " << add_k << " нових елементів в кінець: ";
71      printarr(arr, size);
72
73      delete[] arr;
74
75      return 0;
76  }
```

Рис 19 - 20. код до задачі lab 4

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/lab_4_task_1_variant_10_dariia_sierova.cpp

Завдання №2 Lab# programming: Lab 5

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

bool hasNegative(const int* row, int cols){
    for (int j = 0; j < cols; j++){
        if (row[j] < 0){
            return true; // відємні числа є
        }
    }
    return false; // відємних чисел нема
}

void printMatrix(int** matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

int main() {
    srand(time(0));

    int rows; //рядки
    cout << "Введіть кількість рядків: ";
    cin >> rows;

    int cols; //стовпці
    cout << "Введіть кількість стовпців: ";
    cin >> cols;

    //двовимірний динамічний масив
    int** matrix = new int*[rows];
    for (int i = 0; i < rows; i++){
        matrix[i] = new int[cols]; // адреса масиву(рядок)
        for(int j = 0; j < cols; j++){
            matrix[i][j] = rand()%201 - 100; // -100; 100
        }
    }
    //початкова матриця
    cout << "Початкова матриця: " << endl;
    printMatrix(matrix, rows, cols);
```

```
    int lastRow  = rows - 1; // межа перевірки рядків

    for ( int i = 0; i <= lastRow;){
        if (hasNegative(matrix[i], cols)){ // пошук відємних в і рядку
            // 1. знищуємо рядок
            for(int j = 0; j < cols; j++){
                matrix[i][j] = 0;
            }
            // 2. зберігаємо адресу
            int* temp = matrix[i];

            // 3. зсуваємо вказівники
            for(int k = i; k < lastRow; k++){
                matrix[k] = matrix[k + 1];
            }

            // 4. записуєм збережену адресу в кінець
            matrix[lastRow] = temp;

            // 5.  зменшуєм останню межу перевірки (останній рядок вже обробле
            lastRow--;
        } else {
            // якщо від'ємних ел немає переходим на наступний рядок
            i++;
        }
    }
    cout << "Матриця після обробки:" << endl;
    printMatrix(matrix, rows, cols);

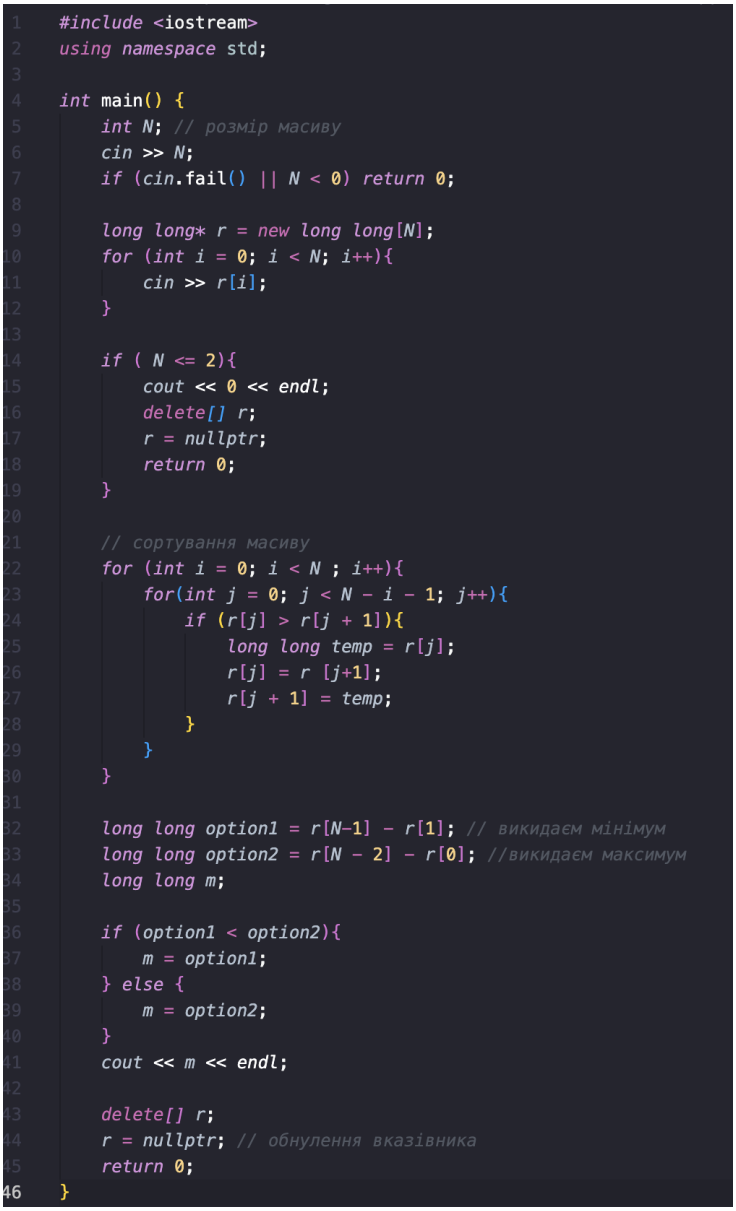
    for (int i = 0; i < rows; i++) {
        delete[] matrix[i]; // спочатку видаляємо кожен рядок
    }
    delete[] matrix; // потім видаляємо масив вказівників

    return 0;
}
```

Рис 21-22. код до задачі lab 5

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/lab_5_task_1_variant_10_dariia_sierova.cpp

Завдання №3 Lab# programming: Algotester Lab 2



```cpp
#include <iostream>
using namespace std;

int main() {
    int N; // розмір масиву
    cin >> N;
    if (cin.fail() || N < 0) return 0;

    long long* r = new long long[N];
    for (int i = 0; i < N; i++){
        cin >> r[i];
    }

    if ( N <= 2){
        cout << 0 << endl;
        delete[] r;
        r = nullptr;
        return 0;
    }

    // сортування масиву
    for (int i = 0; i < N ; i++){
        for(int j = 0; j < N - i - 1; j++){
            if (r[j] > r[j + 1]){
                long long temp = r[j];
                r[j] = r [j+1];
                r[j + 1] = temp;
            }
        }
    }

    long long option1 = r[N-1] - r[1]; // викидаєм мінімум
    long long option2 = r[N - 2] - r[0]; //викидаєм максимум
    long long m;

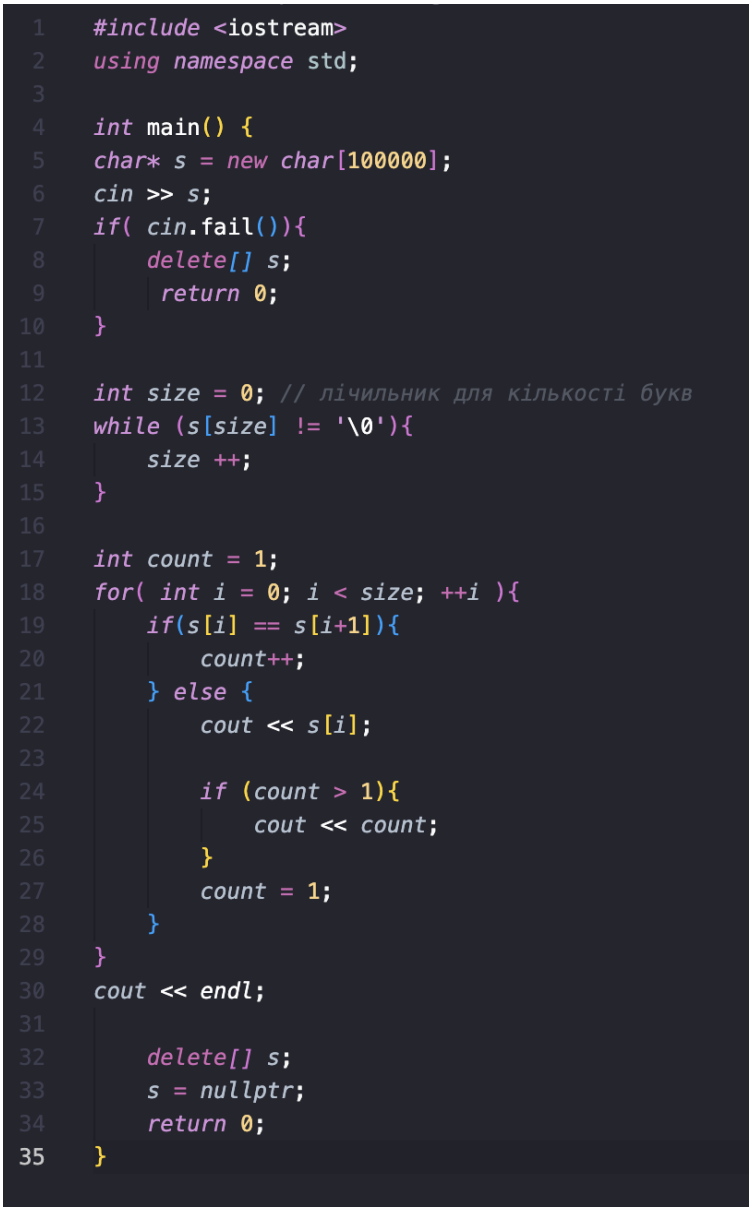
    if (option1 < option2){
        m = option1;
    } else {
        m = option2;
    }
    cout << m << endl;

    delete[] r;
    r = nullptr; // обнулення вказівника
    return 0;
}
```

Рис 23. код до задачі algotester lab 2

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/algotester_task_2_variant_1_dariia_sierova.cpp

Завдання №4 Lab# programming: Algotester Lab 3



```cpp
#include <iostream>
using namespace std;

int main() {
char* s = new char[100000];
cin >> s;
if( cin.fail()){
    delete[] s;
     return 0;
}

int size = 0; // лічильник для кількості букв
while (s[size] != '\0'){
    size ++;
}

int count = 1;
for( int i = 0; i < size; ++i ){
    if(s[i] == s[i+1]){
        count++;
    } else {
        cout << s[i];

        if (count > 1){
            cout << count;
        }
        count = 1;
    }
}
cout << endl;

    delete[] s;
    s = nullptr;
    return 0;
}
```

Рис 24. код до задачі algotester lab 3

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/algotester_task_3_variant_3_dariia_sierova.cpp

Завдання №5 Lab# programming: Class Practice Task

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

const char PALETTE[10] = {' ', '.', ':', '-', '=', '+', '*', '#', '%', '@'};

// очищення буфера після помилки
void clearinput(){
    cin.clear();
    while (cin.get() != '\n'); //ігноруємо все до кінця рядка
}

//звільнення динамічної пам'яті та обнулення вказівника
void freeimage(int**& img, int rows){
    if (img != nullptr){
        //спочатку видаляємо кожен окремий рядок
        for (int i = 0; i < rows; i++){
            delete[] img[i];
        }
        //видаляємо головний масив вказівників
        delete[] img;
        img = nullptr;
    }
}

// виділення динамічної пам'яті під зображення
int** allocateimage(int rows, int cols){
    int** img = new int*[rows];
    //для кожного рядка виділяємо пам'ять
    for (int i = 0; i < rows; i++){
        img[i] = new int[cols];
    }
    return img; // повертаємо адресу головного масиву

}
//1. показати зображення
void showimage(int** img, int rows, int cols){
    //табличний вивід 0–9
    cout << endl << "Зображення " << rows << "x" << cols << endl;
    for (int i = 0; i < rows; i++){
        for( int j = 0; j < cols; j++){
            cout << img[i][j] << " ";
        }
        cout << endl;
    }
    //графічний
```

```
    //графічний
    cout << "Малюнок:" << endl;
    for (int i = 0; i < rows; i++){
        for (int j = 0; j < cols; j++){
            cout << PALETTE[img[i][j]];
        }
        cout << endl;
    }
    cout << "Поточні розміри:" << rows << "x" << cols << endl;
}

//пошук найтемнішого і найсвітлішого
void findextrimes(int** img, int rows, int cols, int &minrow, int &mincol, int &m
    // ввважаєм що перша комірка найбільша і найменша
    minrow = 0; mincol = 0;
    maxrow = 0; maxcol = 0;

    for(int i = 0; i < rows; i++){
        for (int j = 0; j < cols; j++){
            if(img[i][j] < img[minrow][mincol]){
                minrow = i;
                mincol = j;
            }
            if (img[i][j] > img[maxrow][maxcol]){
                maxrow = i;
                maxcol = j;
            }
        }
    }
}

void printstatistics(int** img, int rows, int cols){
    int minrow, mincol, maxrow, maxcol;
        findextrimes(img, rows, cols, minrow, mincol, maxrow, maxcol);

    cout << "\nНайтемніший піксель: " << img[minrow][mincol]
         << " (рядок " << (minrow + 1) << ", стовпець " << (mincol + 1) << ")\n";
        cout << "Найсвітліший піксель: " << img[maxrow][maxcol]
         << " (рядок " << (maxrow + 1) << ", стовпець " << (maxcol + 1) << ")\n";


    double totalsum = 0;
```

```
    //рахуєм заг суму кількість пікселів
    for(int i = 0; i < rows; i++){
        int* row = img[i]; //вказівник на почтаок поточного рядка
        for(int j = 0; j < cols; j++){
            totalsum += *(row + j);
        }
    }

    //середня яскккравість
    double totalavg = totalsum / (rows * cols);
    cout << fixed << setprecision(2) << "Середня яскравість: " << totalavg << end

    //середня яскравість для кожного рядка
    cout << "Середня по рядках: ";
    for (int i = 0; i < rows; i++){
        int* row = img[i]; //переставляєм вказівник на поточний рядок
        double rowsum = 0;
        for(int j = 0; j < cols; j++){
            rowsum += *(row + j);
        }
        cout << fixed << setprecision(2) << (rowsum / cols) << " ";
    }
    cout << endl;
}

//інвертуввання яскравості
void invertbrightness(int** img, int rows, int cols){
    for( int i = 0; i < rows; i++){
        for(int j = 0; j < cols; j++){
            img[i][j] = 9 - img[i][j];
        }
    }
    cout << "\nЯскравість інвертовано.\n";
}

// перестановка ел за адресою
void swapPixels(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```cpp
//віддзеркалення по горизонталі
void fliphorizontal(int** img, int rows, int cols){
    for (int i = 0; i < rows; i++) {
        // цикл йде до середини рядка, інакше пікселі повернуться назад
        for (int j = 0; j < cols / 2; j++) {
            // передаємо адреси лівого пікселя та протилежного правого
            swapPixels(&img[i][j], &img[i][cols - 1 - j]);
        }
    }
    cout << "\nЗображення віддзеркалено.\n";
}

// поворот на 90
int** rotateright(int** img, int rows, int cols) {
    // при повороті ширина стає висотою і тд
    int newrows = cols;
    int newcols = rows;
    //виідялєм память під новий масив
    int** newimg = allocateimage(newrows, newcols);
    //перенесення у новий масив
    for (int i = 0; i < rows; i++){
        for (int j = 0; j < cols; j++){
            //перший рядок початкового зображення стає останнім стовпцем нового
            newimg[j][rows - 1 -i] = img[i][j];
        }
    }
    freeimage(img, rows);
    return newimg;
}

int main() {
    int rows, cols;

    cout << "Висота зображення (1..20): ";
    if (!(cin >> rows) || rows < 1 || rows > 20) {
        cout << "Помилка: Некоректне значення висоти!\n";
        return 1;
    }

    cout << "Ширина зображення (1..20): ";
    if (!(cin >> cols) || cols < 1 || cols > 20) {
        cout << "Помилка: Некоректне значення ширини!\n";
        return 1;
    }
```

```
        // виділення памяті під зображення
        int **img = allocateimage(rows, cols);

        //зчитаування пікселів
        cout << "Яскравість пікселів (0..9):\n";
        for ( int i = 0; i < rows; i++){
            for (int j = 0; j < cols; j++){
                if(!(cin >> img[i][j]) || img[i][j] < 0 || img[i][j] > 9){
                    cout << "Помилка! Піксель виходить за межі." << endl;
                    freeimage(img, rows);
                    return 1;
                }
            }
        }

        int choice;

        do {
            cout << "\n================ МЕНЮ ================\n"
                 << "1. Показати зображення\n"
                 << "2. Статистика яскравості\n"
                 << "3. Інвертувати яскравість\n"
                 << "4. Віддзеркалити по горизонталі\n"
                 << "5. Повернути на 90 градусів праворуч\n"
                 << "0. Вийти\n"
                 << "Оберіть пункт: ";

            if (!(cin >> choice)){
                cout << "Помилка: Введіть числове значення!\n";
                clearinput();
                continue;
            }

            switch (choice) {
                case 1: showimage(img, rows, cols); break;
                case 2: printstatistics(img, rows, cols); break;
                case 3: invertbrightness(img, rows, cols); break;
                case 4: fliphorizontal(img, rows, cols); break;
                case 5: {
                    img = rotateright(img, rows, cols);
                    int temp = rows;
                    rows = cols;
                    cols = temp;
                    cout << "\nЗображення повернуто. Нові розміри: " << rows << " x " << cols << "\n";
                    break;
                }
                case 0:
                    cout << "\nЗавершення роботи...\n";
                    break;
                default:
                    cout << "Невідомий пункт меню! Спробуйте ще раз.\n";
                    break;
            }

        } while (choice !=0);
        // прибираєм память
        freeimage(img, rows);

        return 0;
    }
```

Рис 25-30. код до Class Practice task

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/practice_task_1_dariia_sierova.cpp

Завдання №6 Practice# programming: Self Practice Task 1

```cpp
// решта
#include <iostream>

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    int change = 500 - n;

    int size = 9;
    int* cup = new int[size]{500, 200, 100, 50, 20, 10, 5, 2, 1};


    int total = 0;

    for (int i = 0; i < size; i++) {
        int current = cup[i];
        if (change >= current) {
            total += change / current;
            change %= current;
        }
    }

    cout << total << "\n";

    delete[] cup;
    cup = nullptr;

    return 0;
}
```

Рис 31. Код до задачі Self Practice Task 1

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/self_practice_algotester_task_1_dariia_sierova.cpp

Завдання №7 Practice# programming: Self Practice Task 2

```cpp
//Запити на матриці
#include <iostream>
using namespace std;

void swap(int* a, int* b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main(){
    ios_base::sync_with_stdio(false); //вимикає синхрон. з потоками с і с++
    cin.tie(NULL); //дозволяє cin зчитувати дані без постійного примусового очищення буфера cout

    int rows;
    cin >> rows;
    if(cin.fail() || rows <= 0) return 0;

    int cols;
    cin >> cols;
    if(cin.fail() || cols <= 0) return 0;
    //виділяєм память під кожен ел
    int** matrix = new int*[rows];
    for( int i = 0; i < rows; i++){
        matrix[i] = new int[cols];
    }
    //елементи матриці
    for(int i = 0; i < rows; i++){
        for(int j = 0; j < cols; j++){
            cin >> matrix[i][j];
        }
    }

    int q;
    cin >> q;

    while(q--){
        int type;
        cin >> type;

        if(type == 1){
            int x1, y1, x2, y2, h, w;
            cin >> x1 >> y1 >> x2 >> y2 >> h >> w;

            //перетвооримо на індекси
            x1--; y1--;
            x2--; y2--;

            for(int i = 0; i < h; i++){
                for( int j = 0; j < w; j++){
                    swap(&matrix[x1 + i][y1 + j], &matrix[x2 + i][y2 + j]);
                }
            }
```

```
        } else if(type == 2){
            int x, y;
            cin >> x >> y;

            // перехід до індексів
            x--; y--;

            //значення елемента за координатами
            cout << matrix[x][y] << "\n";
        }
    }

    for (int i = 0; i < rows; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;

    return 0;
}
```

Рис 32-33. Код до задачі Self Practice Task 2

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/self_practice_algotester_task_2_dariia_sierova.cpp

Завдання №8 Practice# programming: Self Practice Task 3

```cpp
//зуби
#include <iostream>
using namespace std;

int main() {
    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    long long* teeth = new long long[n];

    for (int i = 0; i < n; i++) {
        cin >> *(teeth + i);
    }

    int max_len = 0;
    int current_len = 0;

    for (int i = 0; i < n; i++) {
        if (*(teeth + i) >= k) {
            current_len++;
            if (current_len > max_len) {
                max_len = current_len;
            }
        } else {
            current_len = 0;
        }
    }

    cout << max_len << "\n";

    delete[] teeth;
    teeth = nullptr;

    return 0;
}
```

Рис 33. Код до задачі Self Practice Task 3

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/5524db0124f31236abcc8f0b21a9cd5545bbc677/ai_13/daria_sierova/epic_4/self_practice_algotester_task_3_dariia_sierova.cpp

5. Результати виконання завдань, тестування та фактично затрачений час:

Завдання №1 Lab# programming: Lab 4

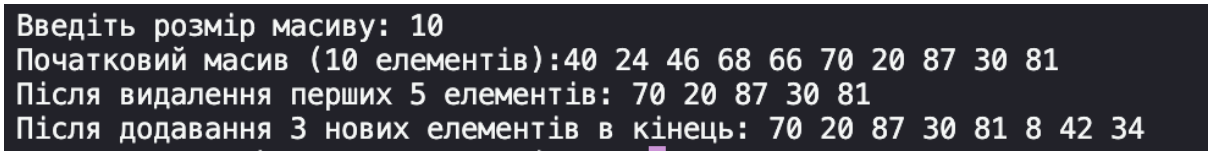


Рис 34. Виконання програми lab 4

Час затрачений на виконання завдання: 25хв

Завдання №2 Lab# programming: Lab 5

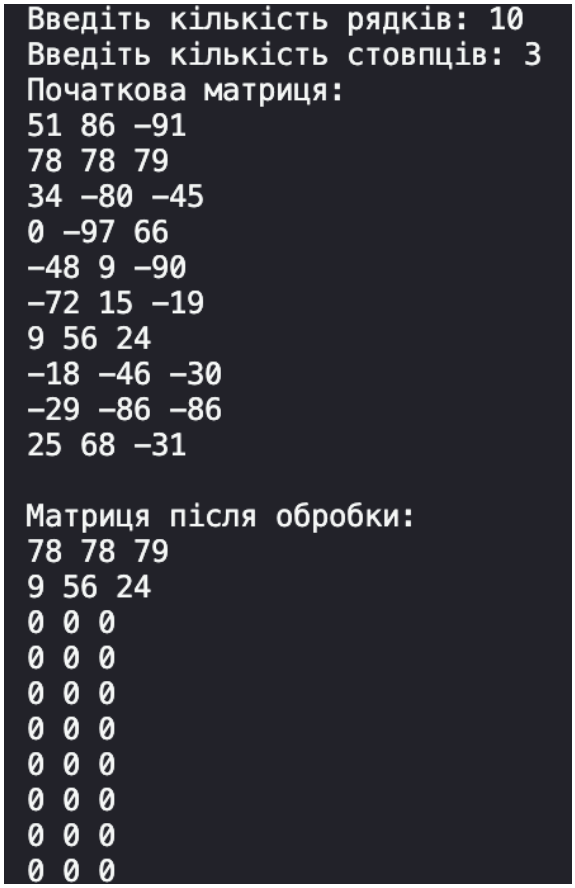


Рис 35. Виконання програми lab 5

Час затрачений на виконання завдання: 40хв

Завдання №3 Lab# programming: Algotester Lab 2



Рис 34. Виконання програми algotester lab 2

Час затрачений на виконання завдання: 25хв

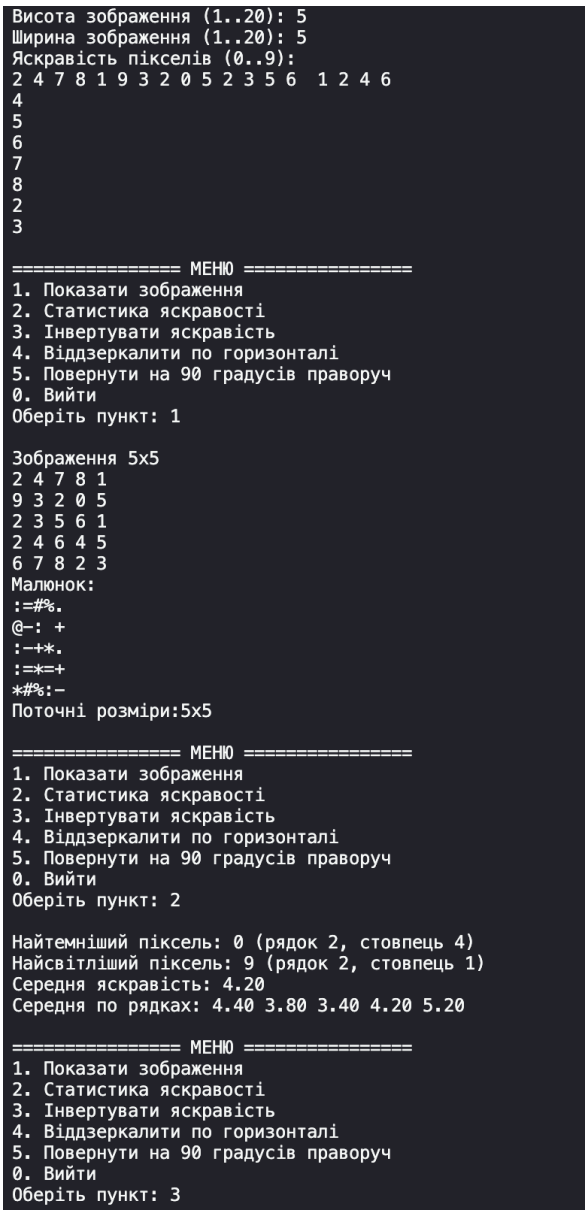
Завдання №4 Lab# programming: Algotester Lab 3



Рис 35. Виконання програми algotester lab 3

Час затрачений на виконання завдання: 40хв

Завдання №5 Practice# programming: Class Practice Task



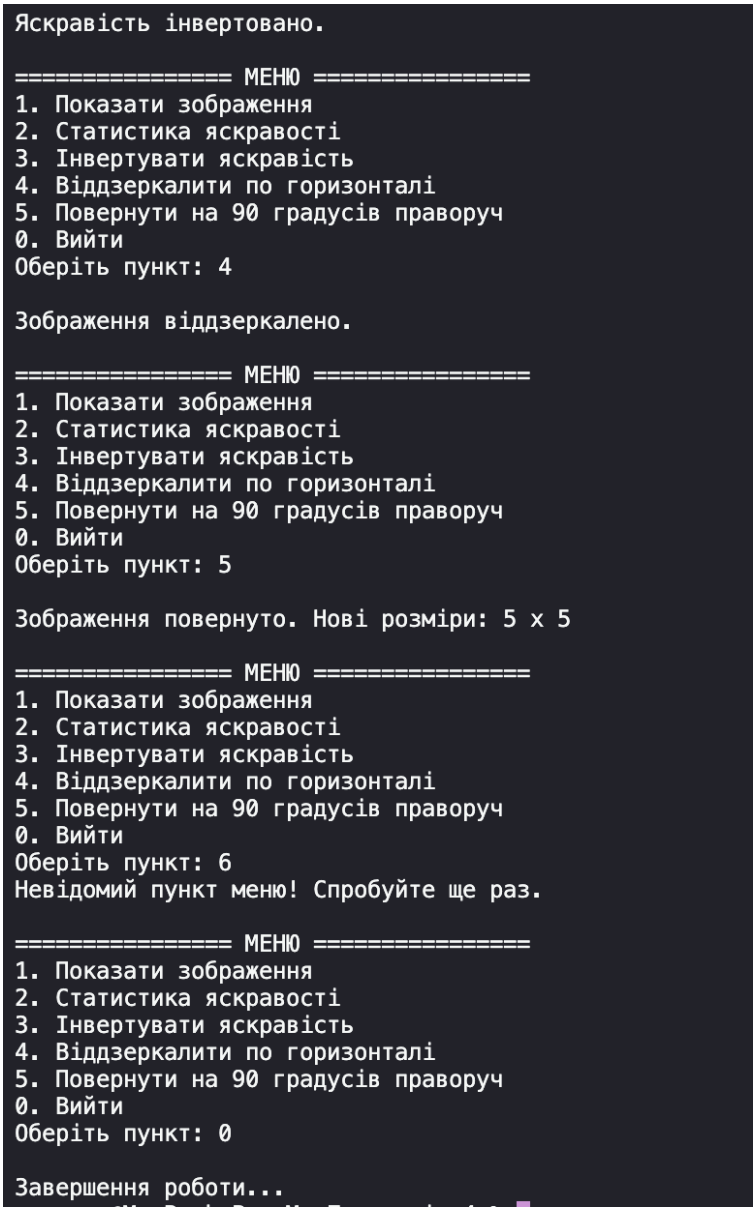


Рис 36. Виконання програми Class Practice task

Час затрачений на виконання завдання: 3 год

Завдання №6 Practice# programming: Self Practice Task 1



Рис 37. Виконання програми self practice task 1

Час затрачений на виконання завдання: 15хв

Завдання №7 Practice# programming: Self Practice Task 2



Рис 38. Виконання програми self practice task 2

Час затрачений на виконання завдання: 15хв

Завдання №8 Practice# programming: Self Practice Task 3



Рис 39. Виконання програми self practice task 3

Час затрачений на виконання завдання: 20хв

6. Кооперація з командою:

- Скрін з 1-ї зустрічі по обговоренню задач Епіку та Скрін прогресу по Трелло



Рис 40. Зустріч з командою 1

- Скрін з 2-ї зустрічі по обговоренню задач Епіку та Скрін прогресу по Трелло

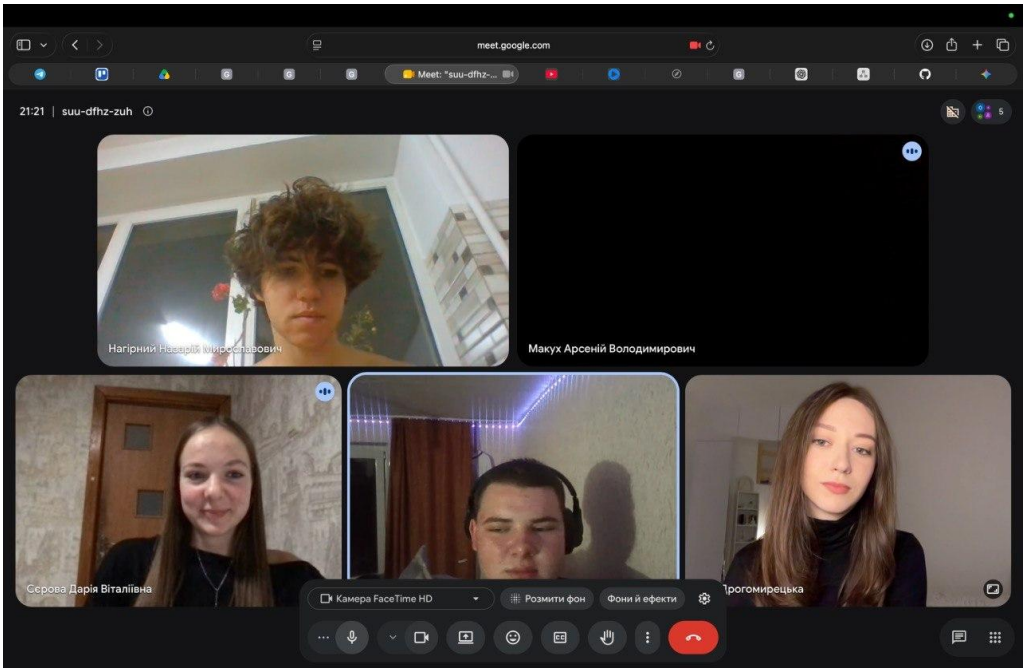


Рис 41. Зустріч з командою 2

- Скрін командної дошки Трелло

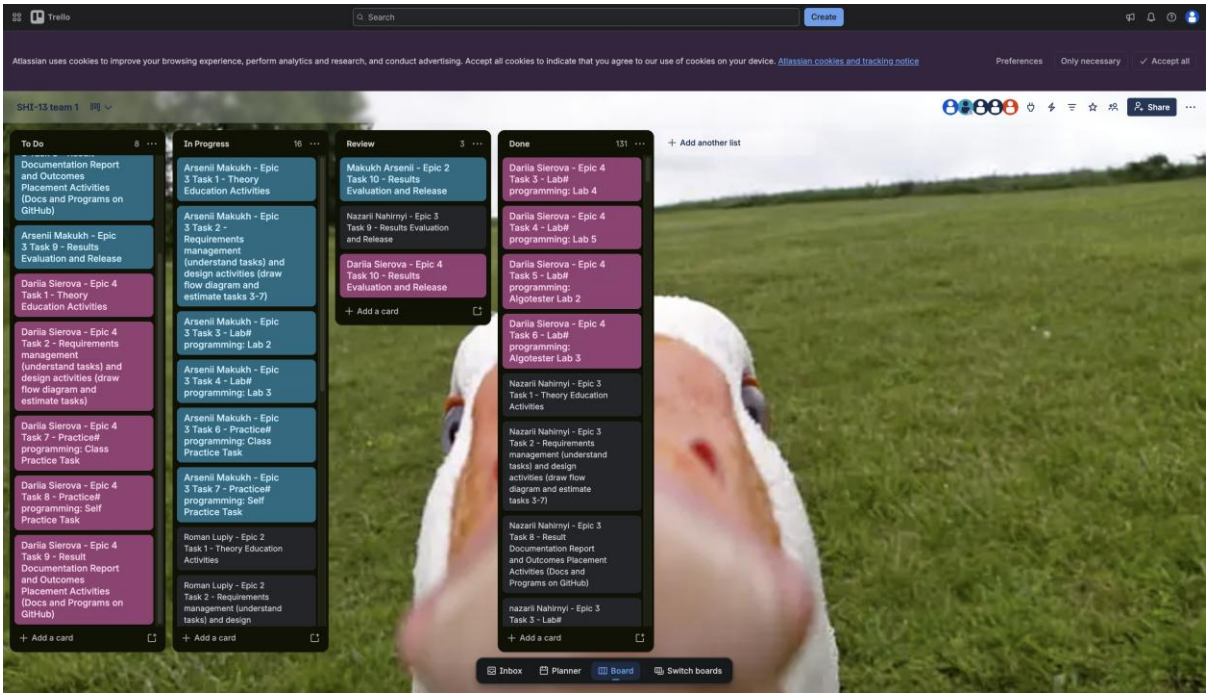


Рис 42. Дошка Трелло

Висновки:

У ході опрацювання теоретичного матеріалу за всіма 6 темами було систематизовано та глибоко засвоєно базові й просунуті концепції мови C++, необхідні для високоефективного програмування та роботи з пам'яттю:

- **Управління пам'яттю та класи пам'яті:** Зрозуміло відмінності між стеком (stack) та купою (heap), розглянуто життєвий цикл і область видимості змінних, що дозволяє оптимізувати використання ресурсів програми.
- **Вказівники, посилання та арифметика вказівників:** Засвоєно роботу з адресами пам'яті (&, *), передачу параметрів у функції за посиланням та вказівником, а також безпосередній доступ до елементів масивів через вказівну арифметику без використання стандартних індексів.
- **Багатовимірні та динамічні масиви:** Опрацьовано створення масивів зі змінною розмірністю в режимі виконання (runtime) за допомогою операторів new та delete[] / delete, дотримуючись правил запобігання витокам пам'яті (memory leaks) та занулення вказівників (nullptr).
- **Алгоритмічна підготовка:** Вивчено та практично закріплено базові алгоритми пошуку, сортування та маніпуляції даними (включаючи жадібні алгоритми та елементи динамічного програмування) для вирішення складних обчислювальних задач.

Отримані теоретичні знання повністю підтверджені практичним застосуванням під час розв'язання алгоритмічних задач та готові для використання в подальших лабораторних і практичних роботах.